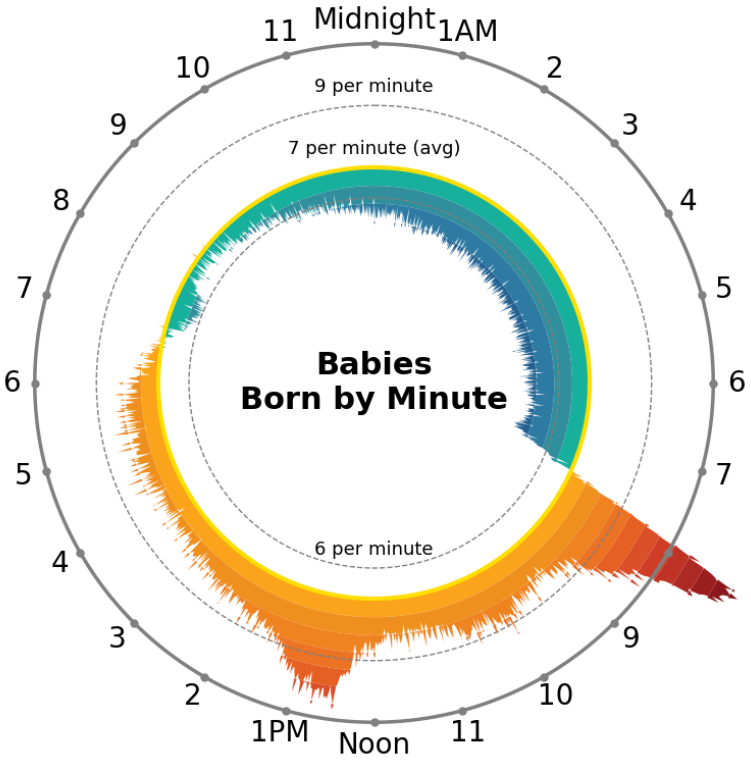
Assignment 4 Report: The Baby Spike

Intro to Data Visualization (Fall 2026)

Group members: Amar Preet Singh, Samridh Barthaval, Prakash Periyasamy

1. Goal

We re-created the Matplotlib version of Nadieh Bremer and Zan Armstrong's *Baby Spike* using the 2022 US birth data. The chart is a clock: time of day goes around the circle and the distance from the centre is the average number of babies born in that minute. A yellow ring marks the yearly average (about 7 per minute) and the space between that ring and the data is filled with stepped blue or orange-red bands. Our result is below and is produced by `Assignment4.ipynb` when run top to bottom.



2. Data preparation

`births.csv` has 3,675,635 rows, one per birth, with a `time` column formatted as `HH:MM`. We sliced out the hour and minute as integers and combined them into a minute of the day from 0 to 1439, then checked that every value falls in that range. Counting births per minute with `value_counts()` gives the total for the whole year, so we divided by 365 to get a typical day. We used `reindex(range(1440))` so every minute is present even if it had no births.

The overall average is the total number of births divided by the minutes in a year: 3,675,635 / (1440 × 365) = **6.99** babies per minute, which matches the ~7 expected in the handout. The busiest minute is 08:03 with 13.69 babies per minute, and the quietest is 05:08 with 4.59.

3. How the code draws the chart

Polar axis set up as a clock. We use `subplot_kw={"projection": "polar"}`, put 0 at the top with `set_theta_zero_location("N")` and make the angle run clockwise with `set_theta_direction(-1)`. Each minute *m* is placed at angle $2\pi \cdot m/1440$. We repeat the first data point at 2π so the shape closes at midnight without a gap.

Stepped colour gradient. The handout says three colour steps equal a 25% change from the average, so each band is `average × 0.25 / 3` ≈ 0.58 babies per minute thick. For band *k* above the average, the inner edge is `average + k·step` and the outer edge is one step further. We fill from the inner edge to the data, with the data clipped to the band:

```
ax.fill_between(theta, band_inner, np.clip(radius, band_inner, band_outer), color=colour)
```

Where the data is below the band, the clipped value equals the inner edge, so the fill has zero height and nothing shows. Where the data passes the band, the band is filled completely. Drawing all 12 orange-red bands on top of each other builds the gradient, with darker colours further from the average. The 5 blue bands work the same way going inwards. The last band of each list has no outer limit (`np.inf`, or 0 for the blue side) so values beyond the colour list still get the darkest colour. The 8 a.m. peak is +96% above the average, so it uses 12 bands and fits the 12 colours given.

Angular axis at 11 babies per minute. Matplotlib always draws the polar spine and angle labels on the outer edge of the plot, so it can't be moved inwards. Following the handout's advice that only the look matters, we hid the default axis with `ax.axis("off")` and drew our own: a grey circle at radius 11, a circle marker for each of the 24 hours, and the hour labels just outside the circle. The radial limit is set to 14 so the 8 a.m. spike (13.7) is not cut off and crosses the axis, as in the original.

Reference lines and labels. The average ring (#FFDE00) and the dashed 6 and 9 per minute rings are full circles drawn with `ax.plot`. We used `zorder` to put the bands at the bottom, then the average line, the dashed rings, and finally the hour axis on top, so the axis stays visible where the spike crosses it. The text labels are placed with `ax.text(angle, radius, ...)` in data coordinates, so they stay in the right place on the circle.

4. Design decisions

- **Radial layout.** A day is periodic: 11:59 p.m. is next to midnight. A circle shows that without the break a straight time axis would have, and it makes the shape of the day easy to read, with a quiet night (blue) and a busy day (orange).
- **Encoding against the average, not zero.** The interesting part is how far each minute is from normal. Filling between the average ring and the data shows that difference directly. The origin is kept at 0, so the distance from the centre is still the real rate.
- **Discrete bands instead of a smooth gradient.** Bands work like contour lines: you can count them to read the size of a change (3 bands = 25%) without needing a colour bar.

- **Diverging colours.** Cool blue below the average and warm orange-red above it separate the two directions clearly, and darker shades mean bigger differences.
- **Leaving out the "8" label.** In the original the spike covers 8 a.m. We removed the label instead of letting the spike partly cover it, which looked cluttered.
- **Rounded average in the label.** The label says "7 per minute (avg)" to match the reference, but the line itself is drawn at the exact value of 6.99.

5. What the chart shows

Births are below average from the evening through the night and early morning, with the fewest around 5 a.m. Right after 8 a.m. there is a sharp spike to almost twice the average, and there is a second smaller peak around noon. These times line up with scheduled births: inductions and planned C-sections are mostly booked for the start of the working day and around midday. This is the main point of the original article by Zan Armstrong.

6. Attribution

- The birth data comes from the National Center for Health Statistics at the CDC (https://www.cdc.gov/nchs/data_access/vitalstatsonline.htm). We used the births.csv file that came with the assignment.
- The design isn't ours. It's *The Baby Spike* by Nadieh Bremer and Zan Armstrong, made for Scientific American (https://www.visualcinnamon.com/portfolio/baby-spike/). We tried to match the Matplotlib version of it shown in the assignment handout.
- The gradient colours, the rule that three bands make a 25% change, and the tip to use fill_between() all come straight from the handout.
- We used the Matplotlib docs on polar axes a lot to work out how to get a clock-style layout.
- Claude was used occasionally as an AI assistant to help identify and debug coding errors during development. All suggestions were reviewed and tested by the team, and we take full responsibility for the final implementation and submitted work.